N-Body Simulation

Abdullah Siraj Khan (31684), Muhammad Yahya (29352),
Rayyan Adil (30556), Omer bin Affan (30497)
Team: RIZZ RISC-V
Department of Computer Science, IBA Karachi

***Abstract*—This report details the implementation of a N-body simulation, evaluating the performance and stability of different numerical methods. For module 1, we implement a naive $O(N^2)$ algorithm using Leapfrog integration and analyze the role of gravitational softening in preventing numerical singularities. Additionally, we optimized the naive version and used a better performing algorithm Barnes-Hut. used octree-based spatial partitioning reducing the computational cost to $O(N \log N)$**

I. Introduction

The N-body problem remains a cornerstone of computational astrophysics. Simulating the gravitational interaction of N particles requires balancing numerical accuracy with computational cost. This report explores the implementation of the naive approach and the transition toward tree-based optimization.

II. Integration Methods: Euler vs. Leapfrog

Numerical integrators approximate the trajectories of particles over discrete time steps Δt.

A. Euler Integration

The Euler method is a first-order, non-symplectic integrator. It updates position and velocity based on instantaneous derivatives: $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t$. This method does not conserve energy, leading to a systematic "spiral-out" error where orbiting bodies gain artificial energy and drift to infinity.

B. Leapfrog Integration

Leapfrog is a second-order symplectic integrator utilizing a Kick-Drift-Kick sequence. By staggering the velocity updates, it ensures time-reversibility and conserves the system's Hamiltonian (energy).

- **Kick**: $\mathbf{v}_{i+1/2} = \mathbf{v}_i + \mathbf{a}_i(\Delta t/2)$
- **Drift**: $\mathbf{r}_{i+1} = \mathbf{r}_i + \mathbf{v}_{i+1/2}\Delta t$
- **Force Update**: Recalculate $\mathbf{a}_{i+1}$ from new positions.
- **Kick**: $\mathbf{v}_{i+1} = \mathbf{v}_{i+1/2} + \mathbf{a}_{i+1}(\Delta t/2)$

Leapfrog is superior for orbital mechanics because its errors are oscillatory rather than cumulative, maintaining stable orbits over large time scales.

III. Naive Algorithm and Softening

A. Time Complexity

The naive algorithm we implemented calculates force by iterating through every unique pair of bodies. For N particles, the number of interactions is $\sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$. So, the time complexity is $O(N^2)$. As N increases, the computation grows quadratically, making it unsuitable for large clusters ($N > 1000$) in interpreted languages like Python.

B. Gravitational Softening

The Newtonian force $F = G\frac{m_1 m_2}{r^2}$ approaches infinity as $r \to 0$. In our simulation if the bodies ever got too close to each other, that would led to a "infinite" force spike that launch the bodies at unrealistic velocities , leading to unpredictable velocities, even potentially throwing them out of there orbits. We use Softening (ϵ) –which keep our simulation safe from the problem of singularity– by using mathematical manipulation:

$$F = G\frac{m_1 m_2}{(r^2 + \epsilon^2)^{3/2}} \tag{1}$$

This ensures forces remain finite, stabilizing high-density interactions.

IV. Barnes-Hut Algorithm

A. Octree Spatial Partitioning

Barnes-Hut optimizes force calculation by recursively partitioning space into an Octree (8 children per node) until each node contains a single particle.

B. Speedup Derivation

Distant groups of particles are treated as a single center of mass if $s/d < \theta$.

- **Naive Operations**: N^2.
- **Barnes-Hut Operations**: $N \log N$.
- **Derivation**: The speedup $S = \frac{O(N^2)}{O(N \log N)} = \frac{N}{\log N}$. For $N = 10,000$, $S \approx \frac{10000}{13} \approx 769$. This logarithmic scaling allows for massive performance gains over the naive method.

V. Implementation Challenges

- **Performance Bottleneck**: Implementing $O(N^2)$ calculations in Python resulted in significant lag for clusters with $N \geq 1000$, showing almost static simulation instead of moving bodies like solar system simulation, which gave hard time for validating our code on real data.
- **Scale Synchronization**: Balancing the gravitational constant G, time step dt, and softening ϵ to accommodate both solar system and star cluster data.

VI. Conclusion

The implementation demonstrates that while naive algorithm is functional for small N, the transition is mandatory for large-scale cosmological modeling such as N-body simulation.

References

[1] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," Nature, vol. 324, 1986.